

Review of “Educational Insights from Code: Mapping Learning Challenges in Object-Oriented Programming through Code-Based Evidence” (Menolli & Strik, arXiv:2507.17743v1)

1. Introduction

This article by André Menolli and Bruno Strik addresses a persistent gap in computing education research: the relationship between students’ conceptual difficulties in object-oriented programming (OOP) and observable deficiencies in their source code. While code smells and SOLID principle violations are well-established in software engineering, the authors note that few studies explicitly link these technical indicators to specific learning challenges. The primary objective is to develop and validate a conceptual map that connects six major OOP learning difficulties (derived from Gutierrez et al., 2022) with concrete code-level issues, thereby enabling educators to diagnose underlying cognitive obstacles beyond mere functional correctness.

2. Methodology

The study employs a qualitative, multi-stage research design. First, the authors selected six OOP learning challenges that can be inferred from source code analysis. Using inductive content analysis, they decomposed each challenge into analysis categories and further into granular codes (e.g., “Method Call Unknown”). Second, they compiled a list of code quality indicators—implementation and design smells, plus SOLID violations—from established literature. Through deductive coding, each indicator was annotated with relevant codes. Finally, the resulting mapping was evaluated by two experienced computer science educators, who applied it to authentic student code excerpts and assessed its pedagogical relevance via structured questionnaires.

3. Results

The principal outcome is a multi-part conceptual map (Figures 6–8) that visually links learning challenges (D02–D07) to analysis categories, codes, and specific code smells or SOLID violations. For instance, “Difficulty in understanding the concept of method” (D03) is associated with smells such as Long Method, Switch Statements, and Duplicated Code. The expert evaluation (Table 4) confirmed that the mapping accurately identifies real classroom difficulties in most cases (e.g., samples S1, S2), while also revealing opportunities for refinement (e.g., capturing the root cause of long parameter lists). Overall, the model demonstrated practical utility as a formative diagnostic tool.

4. Key Insights

Three insights are particularly valuable for my pedagogical and research practice:

First, the explicit linkage between code smells and cognitive misunderstandings. The authors do not treat smells merely as technical debt but as evidence of flawed mental models (e.g., Feature Envy indicating poor encapsulation understanding). This reframes code review in education as a learning diagnostic rather than a quality audit.

Second, the distinction between directly observable and inferable code issues. The map uses dark grey elements to indicate smells that are not directly coded but can be inferred (e.g., God Class from Large Class). This acknowledges that some learning difficulties manifest indirectly, requiring instructors to combine multiple indicators—a useful heuristic for assessment design.

Third, the validation through authentic student code. The expert evaluation with real excerpts (provided in the repository) grounds the model in ecological validity. This convinces me that the map can be operationalised in courses, especially when integrated with static analysis tools to automate initial smell detection.

5. Conclusion

This paper makes a significant contribution by bridging software engineering quality indicators and OOP education research. Its main innovation is the systematic, evidence-based mapping that allows educators to move from “what is wrong with the code” to “what does the student misunderstand.” Future work includes developing automated analysis tools, extending support to languages beyond Java (e.g., Python, C++), and conducting longitudinal studies to measure the impact of early intervention based on the model. The authors also acknowledge limitations, such as potential subjective bias in coding and reliance on instructor familiarity with quality metrics. Overall, the work lays a solid foundation for a code-centric, formative approach to teaching object-oriented programming.